

Building QEX and compiling nf4alphas

Curtis Taylor Peterson

Please follow these steps in the order that they are presented. Getting QEX isn't particularly complicated, as compared to setting up some other lattice gauge theory libraries, but figuring out what went wrong if something goes wrong will be much easier if the steps have been followed in the correct order.

1 Downloading and “installing” Nim

Before we go ahead and log onto the system that we want to run QEX on, we need to download and “install” Nim, the beautiful programming language that QEX is written in. As Nim is a fairly young language with a relatively small user base, it is unlikely that it is available as a module on the system you intend to run on. Lucky for us, all we need to do download pre-built binaries; no building or compilation needed, hence why I've put quotation marks around “install”. To download the pre-installed binaries, open up a browser and go to the following address.

```
https://nim-lang.org/install\_unix.html
```

Now scroll down to **Manual installation** and under that you should see **Pre-build binaries for Linux**. It is likely you'll want the 64-bit version, so download the **x86_64** tarball. Once the tarball has finished downloading, **cd** into the path that it downloaded to and synchronize the tarball up to the system that you wish to run QEX on. For example,

```
cd <path_to_tarball_download>
rsync -avz nim-<a>.<b>.<c>-linux x64.tar.xz <username>@<machine>:<desired_path>
```

where **<a>..<c>** indicates the latest version of Nim; for example, as of the writing of this document, the latest version is 2.2.2, so **<a> = 2, = 2, <c> = 2**. Nim is constantly evolving and being improved upon, so it is unlikely that this will be the version of Nim that you'll be working with, even in the near future. Now you're going to want to **ssh** into the machine you sync'd the Nim tarball up to and extract Nim from the tarball. To do that, execute the following.

```
ssh username@machine
cd <desired_path>
tar -xf nim-<a>.<b>.<c>-linux x64.tar.xz
```

Here **desired_path** is the path that you sync'd the Nim tarball up to. Congratulations, this is, quite literally, all that you need to do to get Nim working on your desired machine. You may now proceed onto the machine-specific steps.

2 System modules and compilers

Note that it is likely you'll have to load certain modules on the system that you are working on for compiling the dependencies that QEX relies upon. I usually prefer to use an Intel compiler, which should be on any system that you are working on. For example, on Fermilab's LQ1 machine, you'd want to load the Intel compiler as follows.

```
module load intel (LQ1)
```

If the system uses Slurm, like most systems do, you can check the list of available modules using `module avail`. For anything else, I recommend consulting the system's corresponding documentation.

3 Building QEX

I am now going to assume that you've downloaded/installed Nim per the instructions in Sec. 1 and you've loaded the appropriate compilers as in Sec. 2. At this point, you should be in `<desired_path>` on the machine you're building QEX on; this is the same path that you unpacked the Nim tarball. If you haven't, `cd <desired_path>`. We are now going to clone my fork of the QEX git repository, as it contains the driver program `nf4alphas` that I wrote for this project. Once you've done that, `cd` into it, and install the QIO/QMP libraries as follows.

```
https://github.com/ctpeterson/qex
cd qex
<desired_path>/qex/bootstrap-travis
```

The `bootstrap-travis` script should compile/install QEX and QMP libraries automatically under `<desired_path>/qex/`. If you `ls <desired_path>/qex/`, you should see both the tarballs for QIO and QMP, along with the folders into which they were installed. You don't have to install both libraries there; if you don't, just make sure that you have the following steps point to the appropriate directories for both libraries.

We're almost done. Now we need to build QEX. With QEX built, we will be able to compile the parts of QEX we need for running Monte Carlo simulations. First, either create your own build directory under `<desired_path>/qex/` and `cd` into it or `cd` into the build directory already under `<desired_path>/qex/build/`. I prefer to create my own build directory because it allows me to easily separate out binary files that may correspond to different compilation choices, such as the SIMD vector length, any restrictions on the SIMD intrinsics QEX should use, the number of colors for the gauge group, and so forth. Let's call this build directory `<build_directory> = <desired_path>/qex/<build_name>/`. To build, execute the following steps.

```
cd <build_directory>
NIM=<desired_path>/nim-<a>.<b>.<c>/bin/nim
export NIM
../configure qmpdir:<path_to_QMP> qioidir:<path_to_QIO>
```

Above, `<path_to_QMP>` and `<path_to_QIO>` are the corresponding paths for QMP and QIO that you installed using `bootstrap-travis`. If you ran `bootstrap-travis` under `<desired_path>/qex/`, then `<path_to_QMP> = <desired_path>/qex/qmp/` and `<path_to_QIO> = <desired_path>/qex/qio/`. Additionally, `<a>`, `<b>`, and `<c>` are placeholders for the version of Nim that you're using, as described in Sec. 1. If the above commands execute successfully, you're done building QEX. Now the last thing that we need to do before compiling `nf4alphas` is to install MDevolve.

4 Installing MDevolve

MDevolve provides QEX the functionalities to run the molecular dynamics with arbitrarily sophisticated integrators. The `nf4alphas` code uses it to run with a 2nd-order Omelyan outer integrator (fermion/Hasenbusch) and a nested 2nd-order Omelyan integrator (gauge). Installing MDevolve is simple. Run the following commands in the directory above where you cloned in the `qex` repository.

```
https://github.com/jxy/MDevolve.git
cd MDevolve
<desired_path>/nim-<a>.<b>.<c>/bin/nimble install
--nim=<desired_path>/nim-<a>.<b>.<c>/bin/nim
```

Note that the last two lines are all one line. What we did here is use Nim's package manager `Nimble` to install MDevolve. Think of `Nimble` as nothing more than `Anaconda` or `pip` for Python; when you "install" a package using `Nimble`, it is kind of like telling a codebase in C++ where a bunch of header files are.

5 Compiling nf4alphas

Now, to compile `nf4alphas`, execute the following commands.

```
cd <build_directory>
make cpp nf4alphas :FUELcompat=1
```

I'm not kidding what I say that's it. You'll of course want to make sure that you have your favorite compiler loaded. This is the same compilation process for any QEX program. Nim compiles to either

C or C++; the above code compiled to C++, since I specified the `cpp`, rather than `c` after `make`. The `:FUELcompat=1` flag tells QEX to compile code that reproduces QHMC (FUEL). I always compile with this flag out of paranoia; note that, if you don't, it does change some of the random number generator code, so it will produce different outputs on single-trajectory runs of the HMC. If you execute `nf4alphas` serially as

```
cd <build_directory>
./bin/nf4alphas
```

from `<build_directory>`, it will run one sample trajectory. See Sec. 7 for a description of running the full program.

6 Customization

You can customize the compilation (adding C flags, tweaking preprocessor directives, etc.) by editing the `qexconfig.nims` file under `<build_directory>`, which was created when you built QEX (Sec. 3). In practice, the only time I ever tweak this file is to tweak the SIMD vector length (`vlen`) or if I want to explicitly tell QEX that it should be utilizing specific SIMD instruction sets (any combination of SSE, AVX, and AVX512 are supported). By default, the SIMD vector length is set to eight (i.e., eight double-precision operations per instruction) and no *specific* intrinsics are chosen (that *doesn't* mean that the default is to have no SIMD vectorization).

7 Running nf4alphas

To run `nf4alphas`, execute

```
<parallel_executor> <parallel_executor_flags> <build_directory>/bin/nf4alphas
--start=<start> --ntraj=<ntraj> --ensemble=<ensemble> --json=<input_file>.json
```

Some explanation:

- `<parallel_executor>`: whatever parallel executor you want to run `nf4alphas` with, say `srun`, `mpirun`, etc.
- `<parallel_executor_flags>`: extra commands for having `<parallel_executor>` allocate resources to `nf4alphas`. There's nothing special here.
- `--start=<start>`: the starting trajectory number
- `--ntraj=<ntraj>`: the number of trajectories to run

- `--ensemble=<ensemble>`: base ensemble name; for example, my convention (borrowed from Oliver Witzel’s convention) is `f<nf>l<l>t<t>b<b>m<m>_<smearing>_<bc>`, where `<nf>` is the flavor number, `<l>` is the spatial length, `<t>` is the temporal length, `<b>` is the bare gauge coupling (no periods), `<m>` is the mass (again, no periods), `<smearing>=HISQ` is the gauge link smearing used in the fermion sector, and `<bc>=pppa` are the boundary conditions. See `/lustre1/nf4alphas/data/Four` on LQ1 for explicit examples.
- `--json=<input_file>.json`: parameter file in JSON format specifying everything else about the simulation (trajectory length, integrator, integrator steps, masses, measurements [including gradient flow], etc.). See `src/alphas/hmc.json` for a sample.

For a full sample of the workflow, see any of the ensemble directories under `/lustre1/nf4alphas/data/Four` on LQ1. You’ll also see a number of helper scripts for analyzing/processing the output of QEX.

Important: In the sample JSON file, you’ll see that the lattice grid, MPI grid, and SIMD grid are explicitly specified. You don’t actually need to specify the MPI and SIMD grids in the JSON file. If you remove them, QEX will try to guess appropriate grids for the run in question. You can also pick and choose anything under “measurements”. Basically, you’ll know if you’ve removed something from the JSON that is explicitly needed by the program; otherwise, you’re safe to remove anything that you want QEX to guess (or which you don’t want measured, specified, etc.).

Finally, if you just run the compiled binary without telling it which JSON file to grab, the program will just run a sample run with default configurations that have been chosen to make testing simple. You can then compare the output against `sample.txt` under `src/alphas`.